GLOBECOM 2025 - 2025 IEEE Global Communications Conference | 979-8-3315-7781-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/GLOBECOM59602.2025.11432412

MiLD: A Microservice Resource Management Framework Based on Latency Decoupling

Chongsong Chen, Junjie Teng, Shijun Ma, Teng Zhong, Yinglei Teng*, *Senior Member, IEEE*
Beijing Key Laboratory of Space-ground Interconnection and Convergence
Beijing University of Posts and Telecommunications (BUPT), Xitucheng Road No.10, Beijing, China, 100876.
Emails: {ccs001110, tengjunjie, mashijun, zhongteng}@bupt.edu.cn, *lilytengtt@gmail.com

***Abstract*—As microservice architecture gains widespread adoption in cloud computing due to its scalability, effective resource management becomes increasingly crucial for cloud service providers. However, evaluating end-to-end (E2E) latency and performing real-time container allocation remains challenging due to the difficulty in decoupling performance impacts caused by service execution variability and complex execution logic. In this work, we propose a lightweight resource scaling framework for microservices based on latency decoupling (MiLD), enabling efficient resource allocation based on container scaling. MiLD recursively traverses the execution time graph and decouples potential linear relationships by analyzing trace data. To tackle the uncertainty brought by the predicative decoupling, we design a Value-at-Risk(VaR) optimization method to mitigate cumulative latency risks, balancing performance and resources. Experiments demonstrate that MiLD can decide in seconds, reducing container usage by 1.3x and tail latency by 2.03x compared to baselines in real microservice applications.**

***Index Terms*—microservices, resource management, link tracing, SLO-targeted.**

I. Introduction

The microservices architecture, characterized by its loose coupling and lightweight nature, has become a preferred choice for modern enterprises, replacing traditional monolithic services. By leveraging cloud computing, this approach provides elastic resources for flexible scaling, while containers enable rapid deployment, significantly enhancing the efficiency of application execution and delivery. End-to-end (E2E) performance is critical as it directly reflects user experience. However, the distributed nature of microservice architectures complicates centralized performance evaluation, posing significant challenges to implementing elastic container-based resource allocation. Consequently, efficient resource allocation strategies that incorporate E2E awareness can reduce cloud tenants' operational costs while ensuring high-quality service.

Recently, studies such as [1], [2] have utilized deep learning techniques to establish the relationship between E2E performance metrics and application provisioning. Nevertheless, these efforts continue to encounter significant challenges. Firstly, decoupling application from independent microservices remains challenging. Changes in microservices (e.g., their number) require recalculating performance models tied to initial configurations. Enterprise applications, often with thousands of microservices [3], use circuit breaking and degradation for availability, causing frequent shifts in microservice combinations and paths. This complicates isolating service performance and inferring E2E performance from inter-microservice relationships.

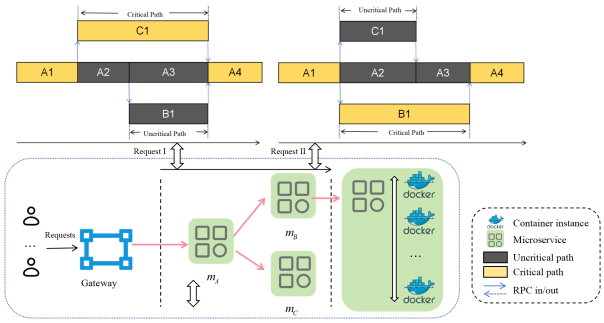


Fig. 1: Critical path impact on execution logic.

Moreover, analyzing microservice performance is challenging due to variable latency influenced by multiple factors. To ensure service quality in cloud environments, providers align resource allocation with Service Level Objectives (SLOs), typically defined by tail latency percentiles (e.g., 95% or 99% of requests meeting a threshold). However, an external metric, E2E latency lacks direct correlation with individual microservice latency. A method to decompose SLOs into components is proposed in [4], yet it complicates tail-percentile SLOs relative to linear metrics and risks underestimating total latency, resulting in inefficiencies and resource waste. Additionally, [5] introduces a method to decompose SLO costs using a probabilistic convolution model, but this approach exhibits high time complexity and requires online calibration.

In link tracing, the change of microservice execution logic is the key reason why it is difficult to decouple application. The critical path, the longest duration among parallel services, drives E2E latency. For example, Fig. 1 shows two execution history graph for microservices A, B, C, the critical path shifts between services B and C based on execution order, reflecting distinct histories of the same chain. Microservice execution varies with invocation timing, and as the order of services B and C shifts, critical paths change—termed variable execution logic. With more microservices and branches, execution complexity increases. Recent studies like [6] model execution logic as sequential chains, an oversimplification for cloud environments. Meanwhile, [5] and [7] analyze hybrid logic (parallel and sequential) in real time to derive E2E

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.

latency from microservice latency, using nonlinear modeling. Yet, frequent logic changes disrupt consistent performance analysis, and these approaches often struggle to explain true service behavior patterns.

To tackle these challenges, we introduce MiLD, a microservice container management framework optimizing E2E latency. The primary objective of this work is to determine the optimal allocation of containers to balance resource use and application performance. Our key contributions are as follows:

- A method to compute E2E weights, estimating each microservice's contribution to E2E latency, addressing execution logic complexity and variability.
- A model capturing service latency uncertainty's impact by formulating resource allocation as a stochastic reward-based portfolio problem based on Value-at-Risk (VaR), quantifying cumulative risk.
- A percentile-based SLO resource allocation strategy using a hybrid difference-of-convex algorithm (HDCA) to optimize performance and resource use in microservices.

II. System Model

A. Microservice-based Application Pattern

In a microservice-based application, each user request is handled by multiple stateless microservices, denoted as $\mathcal{M} = \{1, 2, \ldots, M\}$. Stateful services, such as databases, are excluded from resource allocation, as they prioritize reliable service quality [6]. The container instances usage for stateless MSs are denoted as $\boldsymbol{n} = (n_1, n_2, \ldots, n_M)$, the entry MS, such as an Nginx gateway, triggers multiple request chains $\mathcal{F} = \{1, 2, \ldots, F\}$ for different APIs. For a given $f \in \mathcal{F}$, the E2E latency L_f is determined by the timeline length in the execution history graph and is constrained by the SLO with confidence level γ, denoted as $slo^{\gamma,f}$. The multiple containers of $m \in \mathcal{M}$ follow an M/M/C queueing model, with λ_m representing the request arrival rate at m, and n_m indicating the number of containers. Latency ω_m for each m can be measured using distributed tracing tools such as [8]. In this model, ω_m is positively correlated with the arrival rate λ_m and negatively correlated with n_m. Considering the tail effect of service latency, we assume that ω_m follows an exponential distribution:

$$p(\omega_m = x) = \frac{a_m}{b_m} \exp\left(\frac{a_m}{b_m}(x - c_m)\right), \quad x \geq c_m, \tag{1}$$

$$\begin{aligned} a_m &= u_m^1 n_m + u_m^2, \quad m \in \mathcal{M}, \\ b_m &= u_m^3 \lambda_m + u_m^4, \quad m \in \mathcal{M}, \end{aligned} \tag{2}$$

where c_m is the lower bound of ω_m, and $\frac{a_m}{b_m}$ is the rate parameter of the distribution [9]. The parameters a_m and b_m are affine functions of n_m and λ_m, respectively, determined by $u_m^i > 0, i \in \{1, 2, 3, 4\}$.

During a request, as shown in Fig.1, an execution history graph is generated, with each MS m as a node and directed edges representing request calls. The arrival rate of the entry MS m_0, denoted λ_0, varies over time and can be derived from historical data or predicted using time series methods.

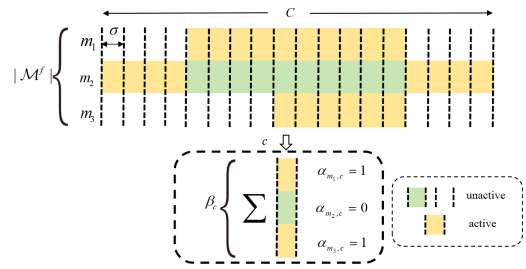


Fig. 2: Decomposition based on the execution history graph.

We estimate arrival rates for all services at the graph level using the Jackson network model [10]. For each MS m, a directed adjacency matrix $\mathbf{T}^r \in \mathbb{R}^{M \times M}$ is extracted in real time from traces for a historical request r. The set of upstream services calling m is denoted $\mathcal{M}_m^u$. Thus, the arrival rate λ_m for m is then computed as

$$\lambda_m = \begin{cases} \lambda_0, & m = m_0, \\ \sum\limits_{m' \in \mathcal{M}_m^u} \mathbb{E}_{r \in \mathcal{R}} \left[\mathbf{T}_{m',m}^r\right] \lambda_{m'}, & \text{otherwise}, \end{cases} \tag{3}$$

where $\mathcal{R}$ represents the set of historical traces.

B. Decoupling E2E Latency in Execution History Graph

To investigate the relationship between E2E latency and individual MSs, we propose a weight-based decoupling method. This method decomposes the execution graph along its time sequence, representing total latency as the sum of individual microservice latencies. For a given execution history graph, we model the linear relationship between L_f and each ω_m, which is not obvious at a macroscopic level. As shown in Fig. 2, we partition the E2E latency in the execution graph into $\mathcal{C} = \{1, 2, \ldots, C\}$, defined as

$$L_f = C\sigma = \sum_{c \in \mathcal{C}} \sigma, \tag{4}$$

where σ represents the length of each time granularity. Let $\alpha_{m,c} \in \{0, 1\}$ denote the working state of MS m at time granularity c, where $\alpha_{m,c} = 1$ indicates that m is active in the c-th time slot. Therefore, ω_m is the sum of $\alpha_{m,c}$ across all time granularities:

$$\omega_m = \sum_{c \in \mathcal{C}} \alpha_{m,c} \sigma. \tag{5}$$

Let β_c denote the number of MSs that are simultaneously in a parallel working state at c. The relationship is given by

$$\beta_c = \sum_{m \in \mathcal{M}} \alpha_{m,c}. \tag{6}$$

Based on (4) to (6) and for a specified f, we can derive the relationship between E2E latency and local latency by

$$L_f = \sum_{m \in \mathcal{M}^f} \frac{1}{\alpha_m} \sum_{c \in \mathcal{C}} \frac{\omega_m \alpha_{m,c}}{\beta_c}, \forall f \in \mathcal{F} \tag{7}$$

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.

where $\alpha_m = \sum_{c\in\mathcal{C}} \alpha_{m,c}$ and the set of MSs involved in f is denoted as $\mathcal{M}^f$. We can observe that (7) decomposes the E2E latency into a weighted summation of ω_m, thus the weight for each m can be expressed as

$$k_{f,m} = \frac{1}{\alpha_m}\sum_{c\in\mathcal{C}} \frac{\alpha_{m,c}}{\beta_c} + \varepsilon, \quad m \in \mathcal{M}^f, \tag{8}$$

when time granularity is fine enough, the unoccupied points at the service duration boundaries can be ignored, resulting in a negligible precision error ε. In dynamic variability execution logic, $k^r_{f,m}$ from a trace r alone is insufficient to represent the linear relationship between ω_m and L_f. To address this, we compute the expected value across multiple execution graphs of the same execution chain, given by $k_{f,m} = \mathbb{E}_{r\in\mathcal{R}}[k^r_{f,m}]$, and estimate the linear relationship by

$$L_f = \sum_{m\in\mathcal{M}^f} k_{f,m}\omega_m, 0 \le k_{f,m} \le 1. \tag{9}$$

III. Problem Formulation

To quantify the uncertainty in service latency, we introduce the VaR function, a concept borrowed from finance that measures the potential latency at a given confidence level $\gamma \in [0,1]$:

$$\mathrm{VaR}_\gamma(X) = \inf\{u : F_X(u) \ge \gamma\} = F_X^{-1}(\gamma), \tag{10}$$

where F_X is the cumulative distribution function (CDF) of the variable X. The fundamental form of the resource allocation problem under percentile-based SLO is given by

$$\begin{aligned} \mathcal{P}_1: \quad & \min_{\boldsymbol{n}} \sum_{m\in\mathcal{M}} n_m g_m, \\ \text{s.t.} \quad & \text{C1: } \mathrm{VaR}_\gamma\left(\sum_{m\in\mathcal{M}^f} k_{f,m}\cdot\omega_m\right) \le \mathrm{SLO}^{\gamma,f}, \\ & \text{C2: } 1 \le n_m \le n_{\max}, \end{aligned} \tag{11}$$

where $n_{\max}$ represents the maximum number of allocatable containers, and g_m is defined as the relative cost of resources allocated to a single container of service m (e.g. 0.5 cores vCPU and 500Mi memory). The objective of $\mathcal{P}_1$ is to ensure that each execution chain maintains its SLO requirements while minimizing the total resource cost.

We further show that $\mathcal{P}_1$ can be formulated as a Markowitz-style portfolio optimization problem based on stochastic returns. According to (1) and (2), the service latency ω_m can be expressed as

$$\omega_m = \frac{b_m}{a_m} z_m + c_m, \quad m \in \mathcal{M}, \tag{12}$$

where z_m is an exponentially distributed random variable with a rate parameter of 1. Let $\boldsymbol{z} = (z_1, z_2, \ldots, z_M)^T$ represents an independent M-dimensional exponential random vector. To linearize ω_m with respect to n_m, we introduce

$$x_m = \frac{b_m}{a_m} = \frac{u^3_m\lambda_m + u^4_m}{u^1_m n_m + u^2_m}, \quad \forall m \in \mathcal{M}. \tag{13}$$

Define $\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f) = \boldsymbol{x}^T\boldsymbol{z}_f$ as the modified composite latency, where $\boldsymbol{z}_f = (k_{f,1}z_1, k_{f,2}z_2, \ldots, k_{f,M}z_M)^T$ represents the random variables derived from the E2E weights in (8), thus L_f can be rewritten as

$$\begin{aligned} L_f(\boldsymbol{n}, \boldsymbol{z}) &= \sum_{m\in\mathcal{M}^f} k_{f,m}\frac{b_m}{a_m} z_m + \sum_{m\in\mathcal{M}^f} k_{f,m}c_m \\ &= \widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f) + \sum_{m\in\mathcal{M}^f} k_{f,m}c_m. \end{aligned} \tag{14}$$

Substituting (15) into constraint C1 of $\mathcal{P}_1$:

$$\begin{aligned} & \mathrm{VaR}_\gamma\left(L_f(\boldsymbol{n}, \boldsymbol{z})\right) \le \mathrm{slo}^{\gamma,f} \\ \Leftrightarrow \quad & \mathrm{VaR}_\gamma\left(\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f) + \sum_{m\in\mathcal{M}^f} k_{f,m}c_m\right) \le \mathrm{slo}^{\gamma,f} \\ \Leftrightarrow \quad & \mathrm{VaR}_\gamma\left(\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f)\right) \le \mathrm{slo}^{\gamma,f} - \sum_{m\in\mathcal{M}^f} k_{f,m}c_m \end{aligned} \tag{15}$$

Thus, $\mathcal{P}_1$ can be transformed into

$$\begin{aligned} \mathcal{P}_2: \quad & \min_x \sum_{m\in\mathcal{M}} \left(\frac{a'_m g_m}{x_m} + b'_m g_m\right), \\ \text{s.t.} \quad & \text{C1}': \mathrm{VaR}_\gamma\left(\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f)\right) \le \varsigma^f, \quad \forall f \in \mathcal{F}, \\ & \text{C2}': \frac{a'_m}{n_{\max} - b'_m} \le x_m \le \frac{a'_m}{1 - b'_m}, \quad \forall m \in \mathcal{M}, \end{aligned} \tag{16}$$

where $\varsigma^f = slo^{a,f} - \sum_{m\in\mathcal{M}} k_{f,m}c_m$, $a'_m = \frac{\mu^3_m\lambda_m + \mu^4_m}{\mu^1_m}$, and $b'_m = -\frac{\mu^2_m}{\mu^1_m}$. In $\mathcal{P}_2$, $b'_m g_m$ is a constant and will be ignored. $\mathcal{P}_2$ is a Markowitz-style portfolio problem with random returns [16], where the VaR constraint in C1 is non-convex.

IV. Resource Allocation for VaR Optimization

A. Discrete Concave Conditional Value-at-Risk

To address $\mathcal{P}_2$, we adopt a method from [16] for combinatorial optimization with VaR constraints. By introducing an auxiliary variable $e \in \mathbb{R}$, the VaR function is expressed as the difference of two Conditional VaR (CVaR) functions at confidence level γ:

$$\begin{aligned} \mathrm{CVaR}_\gamma\left(\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z_f})\right) &= \frac{1}{\gamma}\int_0^\gamma \mathrm{VaR}_u\left(\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z_f})\right) du \\ &= \max_{e\in\mathbb{R}}\left\{e - \frac{1}{\gamma}\mathbb{E}_{\boldsymbol{z}\sim\boldsymbol{z_f}}\left[e - \widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z})\right]_+\right\}, \end{aligned} \tag{17}$$

From (14), it is clear that $\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f)$ is linear in $\boldsymbol{x}$, implying that CVaR exhibits properties like subadditivity and concavity [15]. Let $\mathcal{S} = \{1, 2, \ldots, S\}$ denote the set of all possible scenarios for the random variable $\boldsymbol{z}_f$, where each scenario $s \in \mathcal{S}$ has equal probability $P(s = k) = \frac{1}{S}, \forall k \in \mathcal{S}$. Assuming S is bounded, the realization of $\boldsymbol{z}_f$ is denoted as $\boldsymbol{z}^s_f$. k^* is defined as the scenario index satisfying

$$k^* = \arg\max_k \quad k < S\gamma, \quad \forall s \in \mathcal{S}. \tag{18}$$

At confidence level γ, for computational convenience, we choose S such that $S\gamma \in \mathbb{N}$ and set $k^* = S\gamma - 1$. Given a known $\boldsymbol{x}$, the ordered set $\Psi_f = \{\widetilde{L^s_f}(\boldsymbol{x}, \boldsymbol{z}^s_f) \mid s \in \mathcal{S}\}$ represents the sorted values of $\widetilde{L_f}(\boldsymbol{x}, \boldsymbol{z}_f)$ in ascending order

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.

across all scenarios. Thus, (17) can be rewritten as

$$\begin{aligned}\mathrm{CVaR}_\gamma(\widetilde{L_f}(\boldsymbol{x},\boldsymbol{z}_f)) &= \frac{1}{\gamma}\sum_{s=1}^{k^*+1}\widetilde{L_f^s}(\boldsymbol{x},\boldsymbol{z}_f^s)P(s=k^*+1)\\ &= \frac{1}{k^*+1}\sum_{s=1}^{k^*+1}\widetilde{L_f^s}(\boldsymbol{x},\boldsymbol{z}_f^s)\end{aligned} \tag{19}$$

Combining (17) and (19), the relationship between the CVaR function and the VaR function is given by

$$\begin{aligned}\mathrm{VaR}_\gamma(\widetilde{L_f}(\boldsymbol{x},\boldsymbol{z}_f)) &= (k^*+1)\mathrm{CVaR}_{\frac{k^*+1}{S}}(\widetilde{L_f}(\boldsymbol{x},\boldsymbol{z}_f))\\ &\quad - k^*\mathrm{CVaR}_{\frac{k^*}{S}}(\widetilde{L_f}(\boldsymbol{x},\boldsymbol{z}_f))\\ &= p_f(\boldsymbol{x},\boldsymbol{z}_f) - q_f(\boldsymbol{x},\boldsymbol{z}_f),\end{aligned} \tag{20}$$

where p_f and q_f correspond to the CVaR at confidence levels $\frac{k^*+1}{S}$ and $\frac{k^*}{S}$, respectively. Thus, $\mathcal{P}_2$ can be transformed into

$$\begin{aligned}\mathcal{P}_3: \quad &\min_{\boldsymbol{x}} \quad \sum_{m\in\mathcal{M}}\frac{a'_m g_m}{x_m} + \chi_C(\boldsymbol{x})\\ &+\sum_{f\in\mathcal{F}}\tau_f\left[-q_f(\boldsymbol{x},\boldsymbol{z}_f)+p_f(\boldsymbol{x},\boldsymbol{z}_f)-\varsigma^f\right]_+\\ &\Leftrightarrow \quad \min_{\boldsymbol{x}} \quad g(\boldsymbol{x})-h(\boldsymbol{x})\end{aligned} \tag{21}$$

where:

$$g(\boldsymbol{x}) = \sum_{m\in\mathcal{M}}\frac{a'_m g_m}{x_m} + \chi_C(\boldsymbol{x}) \tag{22}$$

$$+\sum_{f\in\mathcal{F}}\tau_f\left(-p_f(\boldsymbol{x},\boldsymbol{z}_f)+\left[p_f(\boldsymbol{x},\boldsymbol{z}_f)-q_f(\boldsymbol{x},\boldsymbol{z}_f)-\varsigma^f\right]_+\right)$$

$$h(\boldsymbol{x}) = -\sum_{f\in\mathcal{F}}\tau_f p_f(\boldsymbol{x},\boldsymbol{z}_f) \tag{23}$$

$$\chi_C(\boldsymbol{x}) = \begin{cases}0, & \frac{a'_m}{n_{\max}-b'_m}\le x_m\le\frac{a'_m}{1-b'_m},\ \forall m\in\mathcal{M}\\ +\infty, & \text{otherwise}\end{cases} \tag{24}$$

Here, τ_f is a hyperparameter balancing resource usage and the VaR constraint. From (22) and (23), $g(\boldsymbol{x})$ is convex for $\boldsymbol{x}>\boldsymbol{0}$, while $h(\boldsymbol{x})$ is piecewise convex, as detailed in IV-B. Thus, $\mathcal{P}_3$ forms a difference-convex (DC) problem.

B. Hybrid Differenece-of-Convex Algorithm(HDCA)

The HDCA iteratively computes a sequence of approximate solutions $\{\boldsymbol{x}^t : t\in\mathbb{N}\}$ for $g(\boldsymbol{x})$ and $h(\boldsymbol{x})$, ensuring that their difference in (21) decreases with each iteration:

$$g(\boldsymbol{x}^t)-h(\boldsymbol{x}^t) > g(\boldsymbol{x}^{t+1})-h(\boldsymbol{x}^{t+1}), \quad \forall t\in\mathbb{N}. \tag{25}$$

The conjugate of any function $f(\boldsymbol{x})$ w.r.t. $\boldsymbol{y}$ is defined as

$$f^*(y) = \sup_{x\in\mathrm{dom}\,f}\left(\boldsymbol{y}^\top\boldsymbol{x}-f(\boldsymbol{x})\right), \tag{26}$$

and the dual problem of (21) is then formulated as

$$\min: h^*(\boldsymbol{y})-g^*(\boldsymbol{y}). \tag{27}$$

Let $\boldsymbol{y}^t$ denote the intermediate value of $\boldsymbol{y}$ obtained at the t-th iteration. We adopt the Hybrid-DCA proposed in [16] to alternately solve the convex difference problem in $\mathcal{P}_3$:

$$\boldsymbol{y}^t = \arg\min_{\boldsymbol{y}\in\partial h(\boldsymbol{x}^t)}\left\{(\boldsymbol{x}^t)^\top\boldsymbol{y}-g^*(\boldsymbol{y})\right\}, \tag{28}$$

$$\boldsymbol{x}^{t+1} = \arg\min_{\boldsymbol{x}\in\mathbb{R}^M}\left\{g(\boldsymbol{x})-h(\boldsymbol{x}^t)-(\boldsymbol{x}-\boldsymbol{x}^t)^\top\boldsymbol{y}^t\right\}, \tag{29}$$

where ∂f represents the subgradient set of f. For the subproblem in (29), we assume that $\partial h(\boldsymbol{x}^t)$ is unique for certain $\boldsymbol{x}^t$, i.e., $\boldsymbol{x}^t$ is not a discontinuity point and h is differentiable. In this case, the subgradient simplifies to the gradient. For the first k^*+1 elements of the ordered set Ψ_f, the computation is given by

$$\begin{aligned}\nabla h(\boldsymbol{x}^t) &= -(k^*+1)\nabla\sum_{f\in\mathcal{F}}\tau_f p_f(\boldsymbol{x},\boldsymbol{z}_f)\\ &= -\sum_{f\in\mathcal{F}}\sum_{s=1}^{k^*+1}\tau_f\boldsymbol{z}_f^s.\end{aligned} \tag{30}$$

However, elements in Ψ_f may repeat. Let the set of scenarios with identical $\widetilde{L_f^s}$ values be $\tilde{\mathcal{S}} = \{s : s\neq k^*+1,\ \widetilde{L_f^s} = \widetilde{L_f^{k^*+1}},\ s\in\mathcal{S},\ s_f^{\min}\le s\le s_f^{\max}\}$. In (30), the gradient depends on the ordering of Ψ_f. On this basis, a subset $\tilde{\mathcal{S}}_f^*\subset\tilde{\mathcal{S}}$ exists with $\left|\tilde{\mathcal{S}}_f^*\right| = k^*+2-s_f^{\min}$, which can replace $\{s_f^{\min},\ldots,k^*+1\}$ in $\tilde{\mathcal{S}}$, resulting in multiple possible gradients. In this case, $h(\boldsymbol{x})$ is non-differentiable at $\boldsymbol{x}^t$. When $k^*+1<s_f^{\max}$, $\boldsymbol{x}^t$ lies at a breakpoint of a polyhedral convex function, and the subgradient is computed by

$$\begin{aligned}\partial h(\boldsymbol{x}^t) = \Bigg\{&-\sum_{f\in\mathcal{F}}\tau_f\left(\sum_{s=1}^{s_{\min}-1}\boldsymbol{z}_f^s+\sum_{s\in\tilde{\mathcal{S}}_f^*}\boldsymbol{z}_f^s\right):\\ &\forall f\in\mathcal{F},\forall\tilde{\mathcal{S}}_f^*\subset\tilde{\mathcal{S}},|\tilde{\mathcal{S}}_f^*| = k^*+2-s_f^{\min}\Bigg\}.\end{aligned} \tag{31}$$

For the subproblem (29), we reformulate it into as follows

$$\begin{aligned}\min_{\boldsymbol{x}}: &\sum_{m\in\mathcal{M}}\frac{a'_m g_m}{x_m}+\chi_C(\boldsymbol{x})-(\boldsymbol{x}-\boldsymbol{x}^t)^\top\boldsymbol{y}^t\\ &+\sum_{f\in\mathcal{F}}\tau_f\left(-p_f(\boldsymbol{x},\boldsymbol{z}_f)+\left[p_f(\boldsymbol{x},\boldsymbol{z}_f)-q_f(\boldsymbol{x},\boldsymbol{z}_f)-\varsigma^f\right]_+\right)\end{aligned} \tag{32}$$

According to (17), we introduce $\boldsymbol{e}\in\mathbb{R}^{2\times\mathcal{F}}$ to construct p_f and q_f. Furthermore, to handle the nonlinear positive part terms $[\cdot]_+$ in (17) and (32), we introduce auxiliary variables $\boldsymbol{\nu}\in\mathbb{R}^{\mathcal{F}}$ and $\boldsymbol{u}\in\mathbb{R}_+^{2\times S\times\mathcal{F}}$ and reformulate these terms with constraints C3, C4, and C5, thereby expressing (32) as

$$\begin{aligned}\mathcal{P}_4: \min_{\boldsymbol{x},\boldsymbol{\nu},\boldsymbol{u},\boldsymbol{e}} &\sum_{m\in\mathcal{M}}\frac{a'_m g_m}{x_m}+\sum_{f\in\mathcal{F}}\tau_f\nu_f-\boldsymbol{x}^T\boldsymbol{y}^t+\chi_C(\boldsymbol{x})\\ \text{s.t.}\quad &\text{C3: } (k^*+1)e_{p,f}-\sum_{s=1}^{S}u_{p,s,f}\ge-\nu_f, \quad f\in\mathcal{F},\\ &\text{C4: } k^*e_{q,f}-\sum_{s=1}^{S}u_{q,s,f}+\varsigma^f\ge-\nu_f, \quad f\in\mathcal{F},\\ &\text{C5: } u_{\phi,f,s}\ge t_{\phi,f}-\boldsymbol{x}^T z_f^s, \quad f\in\mathcal{F}, s\in\mathcal{S}, \phi\in\{p,q\}.\end{aligned}$$

The resulting problem $\mathcal{P}_4$ is a convex optimization problem. We outline the main procedure of HDCA in Algorithm 1.

V. Evaluation

A. Experiment Setup

Cluster Setup: We deployed Qume virtual machines on two physical hosts equipped with Intel Xeon Gold 6248R CPUs (96 cores, 512GB memory per host) and set up a Kubernetes cluster(v1.23.5).

Load and Traces Setup We configured static loads with an RPS range of [10, 500] and burst-type dynamic loads from

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Resource Scaling Using HDCA

Input: $\mathcal{M}$: Set of microservices, $\mathcal{F}$: Set of functions, $\boldsymbol{\lambda}$: Request rate, ϖ: A small non-zero value to ensure convergence, S: Number of scenarios
Output: $\boldsymbol{x}$: Best container allocation vector

Initialize: Set $\boldsymbol{x}^0 = \mathbf{0}$, $\boldsymbol{y}^0 = \mathbf{0}$, iteration index $t = 0$
for $f \in \mathcal{F}$ **do**
  for $m \in \mathcal{M}$ **do**
    Compute weight $k_{f,m}$ using Eq. (8)
  end for
  Generate $\{\boldsymbol{z}_f^1, \boldsymbol{z}_f^2, \ldots, \boldsymbol{z}_f^S\}$ through sampling
end for
while $(g(\boldsymbol{x}^{t-1}) - h(\boldsymbol{x}^{t-1})) - (g(\boldsymbol{x}^t) - h(\boldsymbol{x}^t)) > \varpi$ **do**
  for $f \in \mathcal{F}$ **do**
    Set $\mathcal{Z}_f \leftarrow \varnothing$
    for $s \in \mathcal{S}$ **do**
      Compute $\tilde{L}^s(\boldsymbol{x}^t, \boldsymbol{z}_f^s)$, $\mathcal{Z}_f \leftarrow \mathcal{Z}_f \cup (\tilde{L}^s, \boldsymbol{z}_f^s)$
    end for
    Sort($\mathcal{Z}_f$) by L^s
  end for
  $\boldsymbol{y}^t \leftarrow \arg\min_{\boldsymbol{y}} \left\{(\boldsymbol{x}^t)^\top \boldsymbol{y} - g^*(\boldsymbol{y}) : \boldsymbol{y} \in \partial h(\boldsymbol{x}^t)\right\}$
  $\boldsymbol{x}^{t+1} \leftarrow$ solve $\mathcal{P}_4$
  $t \leftarrow t + 1$
end while
return $\boldsymbol{x}$

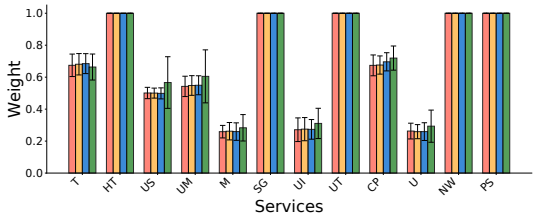
[11], both generated by Locust [12]. The dynamic load was scaled to [200, 800] for our cluster. Distributed tracing was implemented using Jaeger with a 0.05 sampling rate.

Application Setup: MiLD is evaluated using the open-source microservice benchmark DeathStarBench [13], and we use two applications: 1) Social Network, 2) Media Microservice. They consists of over 10 MSs and each container is allocated 0.5 vCPU cores and 512MB memory.

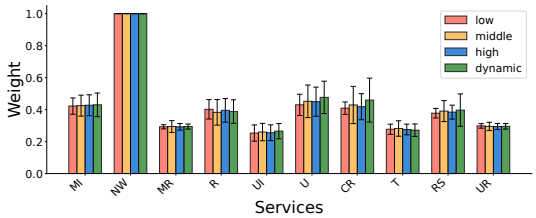
B. Traces Analysis Based on Weight Decoupling Method

This section evaluates the weight decoupling method using real microservice traces. We collect 10^3 traces for each load group, Fig. 3 shows the weight of each service and its variation across traces. The E2E weights of six services and one comment function remain close to 1, indicating they consistently lie on the critical path. Under static load, the average coefficient of variation (CV) is 0.116. With dynamic load, the average CV slightly increases to 0.165, suggesting that load changes have limited impact on E2E weight stability.

For each tracing group, 80% of the data is used for training to generate service weights, while the remaining 20% is used for testing. The weights learned from the training set are applied to the test set to compute the predicted composite latency, which is then compared with the actual E2E latency. The number of latency slices C is set to 10^4 to ensure accuracy. We use RR and MRE as evaluation metrics to assess prediction performance. As shown in Fig. 4, under static load, RR consistently exceeds 0.92, and MRE stays below 5%. Compared to static load, correlation and accuracy under dynamic load drop by 4.5% and 1.7%, respectively. Despite some variability in individual service latencies, the core execution logic of the chain remains stable. These results demonstrate that the proposed weight decomposition method effectively captures variations in execution logic and accurately reflects the composition of E2E latency.

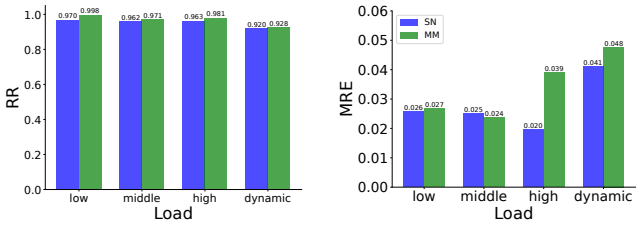


(a) Social network



(b) Media microservices

Fig. 3: E2E Weight statistics across services.



(a) RR (b) MRE

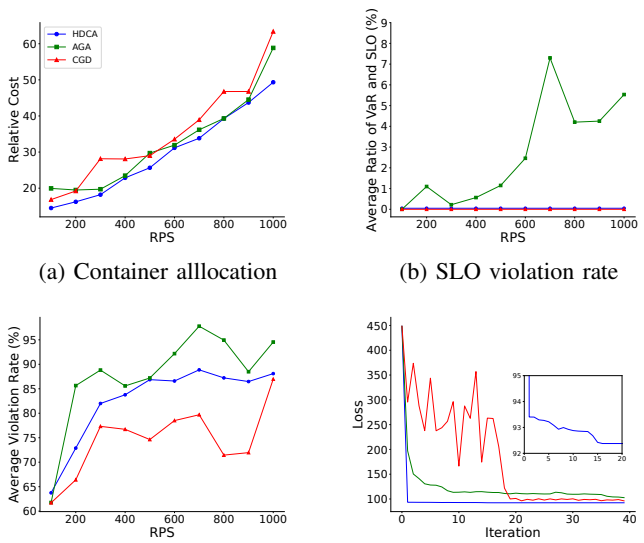
Fig. 4: Composite vs. Real Latency Under Varying Loads.

C. Evaluation of Proposed Container Allocation Method

In this section, we validate the HDCA-based container resource allocation method's effectiveness in balancing cost, performance, and computational efficiency, comparing it with: (1) **Adaptive Genetic Algorithm (AGA)** [14]: Models variables as chromosome genes, updating populations via genetic operations (selection, crossover, mutation) with adaptive annealing and genetic control. (2) **CVaR Substitution and Gradient Descent (CGD)** [15]: Substitutes VaR with CVaR, a convex upper bound, minimizing risk using gradient descent.

Static Load. Consider the case where the tail percentile is set to 0.95, with SLOs for the social network configured at 200ms, 100ms, and 100ms, respectively. Here, S is set to 2×10^3 to control element collisions and balance time complexity. As shown in Fig. 5a, HDCA and AGA perform similarly in resource cost, while CGD performs the worst, with an average resource usage 18.99% higher than HDCA. Fig. 5b shows the average violation rate of the E2E latency

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.



(a) Container alllocation (b) SLO violation rate

(c) SLO utilization rate (d) Algorithm convergence

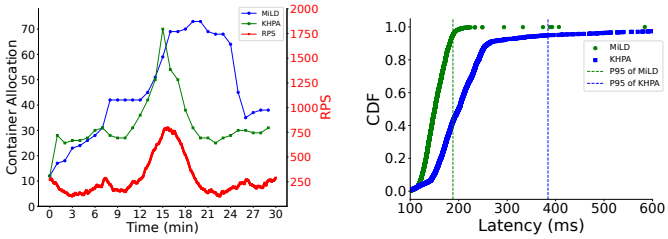
Fig. 5: Resource optimization across methods.

for multiple execution chain SLO. AGA is the only algorithm to exceed the risk value boundary, indicating that heuristic algorithms struggle to control uncertainty risks and fail to meet VaR constraints. Fig. 5c shows utilization, defined as the average ratio of risk latency to SLO, higher values reflect better compliance with VaR constraints. CGD underperforms due to its reliance on the conservative CVaR bound, while HDCA consistently achieves over 80% utilization. Fig. 5d shows the convergence of the three algorithms, with loss computed by (21). HDCA reaches the final value in just 1–2 iterations, faster than AGA (10–20) and CGD (20–25).

Periodic Scaling under Dynamic Load. Fig 6 shows a 30-minute periodic test with a 1-minute interval on MiLD in a social network under the dynamic load provided by [11], comparing it with Kubernetes Horizontal Pod Autoscaler (KHPA) with a CPU threshold of 60%. At the beginning of each decision, MiLD retrieves traces from Jaeger and computes the optimal container allocation through optimization. The average decision time is only 4.5 s. Compared to KHPA, MiLD reduces resource usage by 1.30× and tail latency by 2.03×. As a result, MiLD can significantly adjust the number of containers to adapt to load changes while keeping the tail latency within the SLO limits.

VI. Conclusion

In this paper, we propose MiLD, a lightweight resource provisioning framework for microservices based on link tracing. It integrates centralized performance metrics with distributed observability under varying execution logic by decoupling execution time and estimating service weights at runtime. Using portfolio theory, it combines local latencies into a composite latency and applies a difference-of-convex algorithm for resource allocation. Future work will explore its extension to service monitoring, anomaly detection, and integration with



(a) Containers allocation (b) Latency distribution

Fig. 6: Changes in replicas and request latency in minute-grained execution

machine learning for more efficient microservice resource management.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China (No. U24A20234, No. 62171062), and the National Key RD Program of China (No. 2021YFB3300100).

References

[1] Meng C, Song S, Tong H, et al. DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning[J]. arXiv e-prints, 2023: arXiv: 2309.00859.

[2] Wang Z, Zhu S, Li J, et al. Deepscaling: microservices autoscaling for stable cpu utilization in large scale cloud systems[C]//Proceedings of the 13th Symposium on Cloud Computing. 2022: 16-30.

[3] Luo S, Xu H, Lu C, et al. Characterizing microservice dependency and performance: Alibaba trace analysis[C]//Proceedings of the ACM symposium on cloud computing. 2021: 412-426.

[4] Kannan R S, Subramanian L, Raju A, et al. Grandslam: Guaranteeing slas for jobs in microservices execution frameworks[C]//Proceedings of the Fourteenth EuroSys Conference 2019. 2019: 1-16.

[5] Mirhosseini A, et al. Parslo: A gradient descent-based approach for near-optimal partial slo allotment in microservices[C]//Proceedings of the ACM Symposium on Cloud Computing. 2021: 442-457.

[6] Tong J, Wei M, Pan M, et al. A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network[C]//2021 IEEE International Conference on Web Services (ICWS). IEEE, 2021: 531-540.

[7] Mahgoub A, Yi E B, Shankar K, et al. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs[C]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 2022: 303-320.

[8] Jaeger, "Jaeger: Open Source, Distributed Tracing Platform," Jaeger. [Online]. Available: https://www.jaegertracing.io/.

[9] Luo S, Lin C, Ye K, et al. Optimizing resource management for shared microservices: a scalable system design[J]. ACM Transactions on Computer Systems, 2024, 42(1-2): 1-28.

[10] Jackson J R. Jobshop-like queueing systems[J]. Management science, 1963, 10(1): 131-142.

[11] Wang Z, Li P, Liang C J M, et al. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices[C]//NSDI 24. 2024: 149-165.

[12] Locust, "Locust: A Modern Load Testing Framework," Locust. [Online]. Available: https://locust.io/.

[13] Gan Y, Zhang Y, Cheng D, et al. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems[J]. 2019.

[14] Srinivas M, Patnaik L M. Adaptive probabilities of crossover and mutation in genetic algorithms[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1994, 24(4): 656-667.

[15] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. Journal of risk, 2000, 2: 21-42.

[16] Wozabal D. Value-at-risk optimization using the difference of convex algorithm[J]. OR spectrum, 2012, 34(4): 861-883.

Authorized licensed use limited to: Huazhong University of Science and Technology. Downloaded on April 08,2026 at 02:35:11 UTC from IEEE Xplore. Restrictions apply.